

TrustMesh

Final End-to-End Solution

SIH PS 26125 · Blockchain-Based Secure Platform for Identity, Access Control, and Digital Asset Management

Synthesizes the literal problem statement, the "Sovereign Security Architecture" review, and the 22-section End-to-End Security & Architecture Analysis into one buildable, security-answered solution.

1. Executive Summary

This document is the final, consolidated answer to PS 26125: what to build, in what order, on what stack, and how every security question raised across the prior review and analysis is actually closed — not merely acknowledged.

The problem statement asks for exactly four things: (1) decentralized, cryptographically-verifiable identity independent of a central authority, (2) NFT-based digital assets permanently and verifiably linked to that identity, (3) smart-contract-enforced Role-Based Access Control with Admin/Manager/Auditor/User roles, and (4) an immutable, transparent audit trail of every identity, asset, and access operation. The solution below delivers all four, using DID + Verifiable Credentials for identity, ERC-721 NFTs for assets, on-chain role hashes with expiry for RBAC, and permanent event logging for audit — while correcting the one design gap that would otherwise make the whole platform permanently unusable after ordinary staff turnover: the succession/governance problem identified in the prior review.

Two things are stated plainly up front, because they shape every decision that follows: first, the platform's own contracts previously granted ultimate control to a single address with no path to replace it if all controllers left — this solution fixes that in the contract layer itself, before deployment, because it cannot be fixed after. Second, three specific authorization bugs found in the existing backend (identity hijack via recovery, PII/record poisoning, and ungoverned deletion) are treated as blocking, PO fixes — not backlog items — because they are exploitable today with no special access.

2. Problem Statement Alignment

Each requirement from PS 26125, mapped directly to how this solution satisfies it.

PS 26125 requirement (verbatim intent)	How this solution satisfies it
Decentralized identifier, self-sovereign, cryptographically authenticated, independent of centralized authorities	W3C DID (did:ethr) anchored on-chain; the user's wallet key is the sole controller; authentication is a signed cryptographic challenge, never a password held by any central server
Digital assets as NFTs — unique, traceable, permanently recorded, directly allocated to user identities, cannot be altered or duplicated	ERC-721 AssetNFT contract; every mint/transfer is a governed, Safe-approved transaction; ownership is queried directly from immutable on-chain state, never a mutable database row
Smart contracts govern all operations; only authorized administrators mint and assign NFTs	Every privileged contract function is gated to a Gnosis Safe address, never a single EOA (Externally Owned Account) — 'authorized administrator' is implemented as a multi-party approval group, not one admin key, which is a strictly stronger reading of the requirement
RBAC with Admin, Manager, Auditor, User roles, access rights enforced automatically by smart contracts	AccessControlRegistry (OpenZeppelin AccessControl base) stores roles as bytes32 hashes with an expiry timestamp and revocation status; every privileged backend route re-checks the caller's on-chain role before acting
Every activity — identity creation, NFT creation, allocation, access assignment, transfers, permission updates — immutably recorded, tamper-proof audit trail	Every state-changing contract call emits an event; a durable, reorg-aware indexer (Section 4) projects these into a queryable audit feed; the audit trail is PII-free by construction since PII never touches the chain

What the PS does not specify, and this solution fills in responsibly

- Succession/continuity when administrators change — the PS is silent on this; left unaddressed, it is a fatal design gap (Section 5).
- Key custody and recovery when a user loses their identity's private key — addressed via governed guardian recovery (Section 5).
- What happens to PII if any is collected alongside an identity — addressed via off-chain encrypted storage, since the PS's own "immutable" requirement would otherwise conflict with data-protection law if personal data were put on-chain (Section 6).

3. End-to-End Workflow

The complete operational flow, from a new user's first interaction through ongoing verification. Every write step below is Safe-gated (multi-party approved) by design — no single key executes a privileged action alone.

Step 1 — Identity creation

A new user connects a wallet. The frontend calls `DIDRegistry.registerDID()` directly from the user's own wallet — this is deliberate: the contract binds `msg.sender` as the controller, so identity creation is self-sovereign from the first transaction, not something an admin grants. The backend records the off-chain side (a users row keyed by the DID hash) once the on-chain call is confirmed.

Step 2 — Credential issuance (optional, e.g. KYC/eKYC)

If the platform requires attesting something about the user (identity verification, eligibility), an authorized issuer signs a W3C Verifiable Credential bound to the user's DID. The credential's claims never include raw PII — only attestations (e.g. "identity verified: true"); any underlying personal data goes to the encrypted off-chain vault, referenced only by a content hash if needed.

Step 3 — Role assignment (RBAC)

An Admin proposes a role grant (Admin/Manager/Auditor/User) with an expiry timestamp via `AccessControlRegistry.grantRoleWithExpiry()`. This is not an admin action alone — it is PROPOSED to the governing Safe, and requires the threshold of independent signers (Section 5) to approve on-chain before the role becomes active. Revocation follows the identical governed path.

Step 4 — NFT minting (asset creation)

An authorized Admin/Manager proposes `AssetNFT.mintAsset(to, ipfsCID, contentHash)`. Any document/metadata behind the asset is encrypted client- or server-side BEFORE being uploaded to storage (Section 4) — the mint transaction itself only ever carries a content hash and a storage pointer, never the asset's actual data.

Step 5 — Asset allocation to identity

The mint call's `to`` parameter is the recipient's DID-controlled wallet address — allocation and minting are the same governed transaction, establishing a direct, on-chain, tamper-proof link between the NFT and the recipient's identity at the moment of creation.

Step 6 — Ownership transfer

`AssetNFT.transferAsset()` — governed identically to minting, never an open `transferFrom()` callable by any wallet. This closes a common mistake in NFT-as-asset designs: a real institutional asset should not be freely tradeable by its holder the way a collectible NFT is; transfer requires the same institutional sign-off as the original allocation.

Step 7 — Access verification (read path)

Any authorized verifier queries a public, gated endpoint with a DID; the response is computed live from on-chain state (active role, asset ownership) — never from a potentially stale off-chain cache treated as authoritative, and never returning any PII.

Step 8 — Audit trail

Every event from steps 1–6 is captured by a durable indexer and exposed as a permanent, queryable, PII-free feed — identity creation, role changes, mints, transfers, and permission updates all appear with actor, action, and timestamp, satisfying the PS's transparency requirement directly.

Step 9 — Governance continuity (the step most designs omit)

Officer/administrator turnover is handled by a layered succession mechanism (Section 5) — enrolling a successor is itself a governed, logged event, not an informal handover. This step exists specifically because its absence is what makes an otherwise-correct design permanently fail years into deployment.

4. Detailed Architecture

Layered view

- Client layer: web frontend (citizens, admins) + verifier integration (external organizations) — two different trust levels, two different auth mechanisms (Section 5).
- Edge/Gateway layer: reverse proxy terminating TLS, enforcing mTLS for verifier-organization traffic, rate limiting every request before it reaches application code — nothing unauthenticated reaches the backend directly.
- Application layer: backend API services (identity, credentials, roles, assets, verification, audit) behind a single deny-by-default authentication gate — a new route is protected unless explicitly allowlisted, inverting the previous opt-in-per-route pattern.
- Governance layer: the multi-tier Safe structure (Section 5) — this is the layer that actually authorizes every privileged write; the application layer only proposes, it never executes alone.
- Blockchain layer: the 4 identity/access/asset contracts plus the multisig, deployed on a permissioned network in production (Section 7).
- Off-chain data layer: PostgreSQL for encrypted PII and role-label metadata; a fast cache/session store for hot reads.
- Document/asset storage layer: a private, encrypted, content-addressed store for any file behind an NFT — never public IPFS for anything institution-sensitive.
- Key-management layer: a dedicated secrets/KMS service — no private key or master encryption key ever lives in a plaintext application config file.
- Observability layer: an immutable, append-only application-level audit log (distinct from on-chain events, since it also covers failed attempts and reads) plus infrastructure monitoring.

Component interaction (text diagram)

User Wallet → [signs login challenge] → API Gateway (TLS/mTLS/rate-limit) → Backend Auth Middleware (deny-by-default) → Domain Service (identity/roles/assets) → Safe Proposal → Governance Tiers (Section 5) approve → Smart Contract state change → Event emitted → Indexer projects to audit/verification tables → Verifier/Audit API reads projected state (never replays raw history per request).

5. Governance, Multisig & Succession

EDITORIAL CORRECTION (added after the 2026-09-09 security audit): the 4-tier governance succession model described in this section is a **DESIGN PROPOSAL**, not yet implemented in the current codebase. A full search of every contract, chaincode, and backend source file in the repository confirms no Tier 0/1/2/3 succession module, timelock, or dormant-root key exists anywhere in the shipped system.

The platform's actually-shipped governance control is the Hyperledger Fabric 2-of-3 organizational approval model (chaincode-enforced propose → approve → execute, requiring two of three organizations' real MSP identities), described in docs/ARCHITECTURE.md and docs/API.md. See docs/PROGRESS_LOG.md for current implementation status. The remainder of this section should be read as future design intent, not a description of what is running today.

This section directly answers the most severe issue identified across both prior documents: what happens when the people controlling the system change, and how the platform is prevented from becoming permanently unusable.

The problem being solved

A single admin address (even a well-designed multisig) that has no path to be replaced is a single point of **PERMANENT** failure once its holders are all gone — and given real institutions rotate staff every 2-3 years, this is a certainty over any multi-year deployment, not an edge case.

The 4-tier layered authority model

Tier	Composition	Can do	Cannot do
Tier 0 — Operational	2-of-3 (recommended: 2-of-4 for spare capacity)	Grant/revoke roles, mint/transfer assets — day-to-day governed actions	Replace itself once quorum is lost
Tier 1 — Custodian	3-of-5, seats across separate institutions/departments	Add, swap, or remove Tier 0 signers — succession only	Grant a role, mint an asset, or touch any user data — structurally, not by policy
Tier 2 — Timelock	A contract, held by nobody	Delay every Tier 1 succession action by a fixed public window (e.g. 72 hours), visible on-chain	Originate any action of its own
Tier 3 — Dormant root	Offline key, dual physical custody, hardware-backed	Rotate the Tier 1 Custodian set, but only after a long total-inactivity period (e.g. 180 days)	Do anything at all while the system shows any Tier 0/1 activity

Why the timelock matters: without it, giving Tier 1 the power to replace Tier 0 signers would just mean handing Tier 1 the whole system. With it, a compromised or coerced Tier 1 can only **PROPOSE** a takeover, which sits publicly visible while Tier 0 can cancel it — turning a seizure attempt into a 3-day public event instead of a single silent transaction.

Structural invariants — cheap, must be in place before deployment

- The access-control contract's constructor grants ultimate admin authority to the Succession Module as well as the operational Safe, not to the operational Safe alone — this single change removes the entire single-point-of-failure problem, and because the contract is immutable, it is **ONLY** possible before go-live.
- Enforce owner-count headroom above the approval threshold (e.g. never allow owner count to drop to exactly the threshold) so a single additional departure cannot immediately freeze the system.

- Enroll a successor before a predecessor formally leaves, tied into the institution's existing staff-transfer process — turning succession from a cliff into a months-long, tracked warning.
- Run a quarterly liveness check for every signer — a signer who has lost a key or quietly left is caught in a routine drill, not during an actual incident.

Making credentials outlive their issuer

- Issue credentials from an institutional identity (the office, not one official's hot key) whose identity document retains historical keys with validity windows — so verification checks the credential's issue date against the key that was valid then, not only the current key.
- Anchor batches of issuances as a periodic Merkle root rather than relying solely on a live signature — a credential's validity becomes provable from an immutable root plus an inclusion proof, independent of any single key's continued existence.

6. Identity, Privacy & Data Protection

Q: Where does personal data live, if the PS wants everything 'immutably recorded'?

A: A clear line is drawn: the chain immutably records PROOFS (identity anchors, role hashes, asset ownership, event history) — never PERSONAL DATA. Anything personal (name, ID numbers, contact details, documents) is stored off-chain, field-level encrypted, and referenced on-chain only by a hash. This satisfies 'immutable and transparent' for governance events while remaining compliant with data-protection law, which requires personal data to be correctable/erasable — something a blockchain structurally cannot do.

Q: How is erasure handled if the chain can't delete anything?

A: By destroying the encryption key protecting that specific data, not by touching the chain. The data becomes permanently unrecoverable (the same effect as deletion), while the on-chain hash pointer simply becomes a reference to nothing — the chain itself is never rewritten.

Q: How are private keys protected — for users, and for the system's own operational keys?

A: User keys: held client-side only (in the user's own wallet), never touch any server. System's operational keys (relayer, encryption master keys): must live in a dedicated key-management/HSM service (Section 8) — never in a plaintext configuration file, which was a specific gap identified in the prior review and is treated here as non-negotiable for production.

Q: What happens if a user loses their private key?

A: Governed guardian-based social recovery: the user's designated guardians vote off-chain; once a threshold is met, a single on-chain re-binding transaction executes. Critically, adding a guardian must require the CURRENT key-holder's own active approval — allowing anyone to add themselves as a 'guardian' for someone else's identity (a specific bug found in the prior review) would turn a recovery feature into an identity-theft tool, and this design explicitly closes that gap by construction, plus adds a time-delayed cancellation window so a fraudulent recovery attempt can be caught before it executes.

7. Technology Stack — and Why

Layer	Technology	Why this choice	Security issue it directly solves
Identity	W3C DID (did:ethr) + Verifiable Credentials (did-jwt-vc)	Standards-based, self-sovereign — identity is never a raw wallet address or an NFT, avoiding permanent public exposure of who owns what	Removes reliance on any centralized identity database as a single point of failure/breach — directly answers the PS's 'independent of centralized authorities' requirement
Smart contracts (prototype)	Solidity ^0.8.24, OpenZeppelin AccessControl + ERC-721, Hardhat	Audited base libraries minimize custom cryptographic surface area; fast iteration for proving the design	Reduces smart-contract vulnerability surface by building on battle-tested, previously-audited code rather than custom access-control logic
Blockchain network (production)	Permissioned Hyperledger Fabric (chaincode replacing the prototype's Solidity contracts)	MSP-based identity matches real institutional PKI; channel-based privacy allows different departments/organizations to transact without exposing all data to every participant; no gas-fee model removes an unbounded recurring cost; consensus is configurable among known, accountable participants	Removes the entire class of public-chain risks (51%-style attacks, open mempool front-running, unpredictable throughput/cost) that a public network like the prototype's testnet inherently carries
Multisig governance	Gnosis Safe (vendored contract source) for the prototype; a 4-tier authority model (Section 5) for production	Real, on-chain-enforced multi-party approval — not a policy convention	Directly answers 'only authorized administrators' in the PS, implemented as a group that cannot act unilaterally, plus solves the succession/bricking problem the literal PS text does not anticipate
Backend API	Node.js + Express + TypeScript, deny-by-default auth middleware, schema validation on every input	Fast to build securely, strong typing catches classes of bugs at compile time	Closes opt-in-authentication and unvalidated-input classes of vulnerability found in the prior review
Database	PostgreSQL, field-level AES-256-GCM envelope encryption, additional-authenticated-data binding per record	Mature, ACID-compliant, supports the partitioning and indexing needed at scale	Ensures a raw database compromise still yields only unreadable ciphertext, and prevents ciphertext rows from being silently swapped between different users' records
Cache / sessions	Valkey (open, BSD-licensed Redis-compatible successor)	Native TTL support for session expiry; avoids depending on Redis's post-2024 non-free license	Removes unbounded session/nonce table growth and gives verification reads a fast, bounded-cost path instead of replaying full history per request
Document/asset storage	A private, permissioned, content-addressed store (e.g. a self-hosted IPFS/Kubo cluster)	Keeps the content-addressing model (a document's ID is derived from its own content, proving integrity) while keeping	Prevents any sensitive document from leaving institutional control, and

	— not public IPFS or a public pinning SaaS	data inside institution-controlled infrastructure	removes reliance on a single third-party bearer token as the only thing standing between the internet and every stored file
Encryption (documents & PII)	AES-256-GCM, applied client/server-side BEFORE any upload	Authenticated encryption — detects tampering, not just conceals content; already the correct pattern used for PII, extended uniformly to documents	Directly closes the specific gap where a raw file could otherwise be uploaded to public storage unencrypted
Key management	A dedicated secrets/KMS service (e.g. an open-source Vault-compatible engine), optionally backed by a hardware security module	Keeps master keys out of application memory and out of any plaintext config file entirely; enables key rotation without full re-encryption downtime	Removes the single most damaging finding in the prior review: encryption keys sitting in plaintext next to the data they protect, which makes the encryption itself meaningless if that config file is ever exposed
API Gateway	nginx/Envoy — TLS termination, mTLS for verifier organizations, rate limiting	Standard, mature, puts as much of the security boundary into infrastructure as possible, before any request reaches application code	Closes denial-of-service exposure and removes the need for every route to individually remember to authenticate — nothing unauthenticated reaches the backend at all
Frontend	Next.js + wallet-connect libraries (wagmi/RainbowKit)	Standards-based wallet interaction, broad wallet compatibility	Ensures the user's private key never leaves their own device/wallet extension — the frontend never handles or transmits a raw private key
Audit logging	An immutable, append-only application-level log, separate from on-chain events	On-chain events only capture successful state changes — they miss failed authorization attempts and read-only access, which matter just as much for real audit	Provides the complete audit trail the PS requires, including the security-relevant events a purely on-chain log would silently omit

8. Security Questions — Answered

A direct, consolidated answer to every security question raised across this whole review process.

Governance & succession

Q: What happens when all administrators/signers leave?

A: With the 4-tier model (Section 5) in place, the Tier-3 dormant root can restore governance after a long inactivity period — the system is never permanently bricked. Without this fix (the original design), the system becomes permanently ungovernable, which is exactly why this fix is treated as a pre-deployment blocker, not a later enhancement.

Q: Can 2 colluding signers abuse the system?

A: Yes — no threshold-based system fully prevents collusion by definition. Mitigation is organizational: signers must be drawn from separate institutions/reporting lines so collusion requires coordinated fraud across independent parties, and every approval is permanently, publicly logged, creating real accountability rather than prevention.

Q: Can multisig be rotated without invalidating historical records?

A: Yes — changing who the current signers are never alters or requires touching any past approved transaction; all history remains permanently, immutably logged regardless of the current signer set.

Identity & recovery

Q: Is a wallet the identity, the login mechanism, or the signing mechanism?

A: All three, and this is named explicitly rather than left implicit: the wallet address anchors the DID (identity), signing a login challenge proves control of it (authentication), and signing a governance approval authorizes a specific action (transaction signing).

Q: What if a user's device is compromised?

A: This remains an open, industry-wide risk for any wallet-based system — not something any design fully solves. Mitigation: guardian recovery as the remedy path, hardware-wallet support for high-value/institutional roles, and a custodial onboarding option for citizens with low technical literacy so they are not forced to self-manage a raw key at all.

Q: Can guardians steal a user's identity?

A: Only if (a) they collude to reach the recovery threshold, AND (b) guardian addition isn't properly gated behind the current owner's own approval. This design requires (b) by construction — nobody can add themselves as someone else's guardian — closing what would otherwise be an active identity-theft path, not just a theoretical collusion risk.

Cryptography & storage

Q: Is a blockchain hash the same as security/encryption?

A: No — a hash is an integrity mechanism (proves content hasn't changed); it provides zero confidentiality. Confidentiality comes only from encryption, applied before anything reaches chain-referenced storage.

Q: Does IPFS or a pinning service provide encryption?

A: No. Public content-addressed storage provides integrity and availability, not privacy — anyone with a content identifier can fetch the raw bytes. Encryption must happen at the application layer, before upload, every time, with the platform (not the storage provider) controlling the key.

Q: What happens if a document's storage identifier becomes public?

A: If encrypted correctly beforehand, an attacker who fetches the raw content gets only unreadable ciphertext — having the identifier alone gains them nothing without the key.

Q: How is data revoked from a content-addressed, technically-immutable store?

A: By using a PRIVATE, permissioned store under institutional control (not public IPFS) so content never leaves controlled infrastructure in the first place, and by treating key destruction — not attempted deletion of a replicated blob — as the actual revocation mechanism.

Scale & availability

Q: Does writing every single action directly to the blockchain scale to millions of users?

A: No, not as individual transactions — this was a specific, confirmed failure mode in the prior review (an indexer that exhausts memory, and per-transaction costs that become an unbounded recurring budget liability). The fix: batch periods of activity into a single anchored summary (a Merkle root) rather than one transaction per event, collapsing potentially millions of writes into a handful of anchoring transactions per day, with individual proofs served off-chain on demand.

Q: Is the verification/lookup path fast at scale?

A: Only if it reads from a maintained, indexed projection of chain state — not by replaying the platform's entire history on every single request, which was another confirmed failure mode. The architecture in Section 4 specifically separates 'write to chain' from 'serve fast reads' via a durable indexer.

Compliance & assurance

Q: Is this actually production-ready as designed in the original problem statement's literal wording?

A: Not without the governance/succession fix (Section 5) and the closure of the three specific authorization bugs identified in the prior review (identity hijack via recovery, PII/record poisoning via unvalidated identifiers, and ungoverned single-session deletion). This solution treats those as P0, pre-deployment blockers — everything else in this document assumes they are fixed first.

Q: What outside verification does a real deployment need, beyond this design?

A: Independent, professional security audit of the smart contracts (unit-test coverage is not equivalent to an audit), and — for any government-adjacent deployment — the relevant national security-certification process before go-live. No system should rely solely on its own team's assurance that it is secure.

9. Roles — as Specified in the Problem Statement

The PS names exactly 4 roles; this solution implements them as on-chain, expiring, revocable role hashes rather than permanent labels.

Role	Permissions	Key property enforced
Admin	Grants/revokes roles; proposes governed changes to the Safe/governance layer	Never acts alone — every Admin action is a PROPOSAL requiring multi-party approval, per Section 5
Manager	Proposes NFT minting/allocation and asset-related operations	Same governed-proposal pattern as Admin, scoped to asset operations
Auditor	Read-only access to the full audit trail and system logs	Zero write access anywhere in the system, by design, not by convention
User	Holds a DID, can view own identity/assets, can initiate recovery/guardian setup for themselves	No privileged action; interacts with the system entirely through self-sovereign identity operations

- All roles expire automatically (a configured timestamp, not a permanent grant) and can be revoked instantly — closing the common real-world failure where a departed staff member's access is simply forgotten rather than formally removed.

10. On-Chain vs. Off-Chain Data Model

Data	ON-CHAIN	OFF-CHAIN (encrypted)
Identity	DID + current controller address	—
Roles	Role hash, expiry timestamp, revocation status	Human-readable role ↔ department label (for internal admin UI only)
Credentials	Revocation status / issuance anchor	The signed credential content itself (held by the user)
Personal data	Never	AES-256-GCM encrypted, per-record authenticated binding
Digital assets (NFTs)	Token ID, current owner DID, content hash of metadata	The actual metadata/document, encrypted, in the private content-addressed store
Activity history	Immutable event log — who, what, when (hash-level)	A queryable, indexed projection of the same events for fast lookup

11. Implementation Roadmap

Phase	Focus	Must happen before deployment?
P0 — Days	Fix the 3 live authorization bugs; move to deny-by-default authentication	Yes — trivially exploitable today
P1 — 2-3 weeks	Perimeter hardening: rate limiting, input validation, secure sessions, sanitized error handling	Strongly recommended before any real-network exposure
P2 — 3-4 weeks	The 4-tier governance/succession model, built into the contract layer from the start	YES — the only change in this roadmap that is impossible to retrofit after deployment, since the contract is immutable
P3 — 4-6 weeks	Move document storage to a private, institution-controlled store; move all keys into a dedicated KMS/HSM	Required before handling any real institutional documents or production keys
P4 — 6-8 weeks	Data-layer rewrite for scale: durable indexing, batched/anchored writes, tuned database indexing	Required before onboarding population-scale user volumes, not before an initial pilot
P5 — Ongoing, book early	Independent security audit (contracts + application), CI test coverage, threat-model documentation, incident-response and disaster-recovery drills	Audit must be booked at the very start — it is externally scheduled and slow, and go-live should not proceed without it

Sequencing note: P0 and P1 are pure application-code changes and can start immediately. P2 must land before any real-network deployment — it is the one item that becomes effectively impossible to add later without migrating every user to a new identity registry. P3-P4 can run in parallel with P2 if the infrastructure and contract teams are separate. P5 should be booked from day one, not scheduled at the end.

12. Final Summary

This solution satisfies PS 26125's literal requirements — decentralized identity, NFT-based verifiable asset ownership, smart-contract-enforced RBAC with the 4 named roles, and an immutable, transparent audit trail — using a stack chosen specifically to close every security question raised during this review, not merely to satisfy the wording of the problem statement.

The single most important discipline in this design is treating 'authorized administrators' and 'smart-contract governance' as requiring genuine multi-party approval with a built-in succession path — not a single admin key, and not a multisig with no way to replace itself. Every other control in this document (encryption, key management, scale architecture, audit logging) matters, but this one determines whether the system is still governable ten years from now, which is the actual, practical measure of whether a government- or institution-grade identity platform succeeds.